CODEX 48 / ROUND 3 FINAL SUBMISSION

BrandLens

Evidence-led review and approval for marketing content

PARTICIPANT	TEAM	ROLE
Hariom Thakur	**Solo team**	Product, full-stack development, testing and documentation

PROJECT LINKS

Open the product and code

Demo: brandlens-round2.onrender.com
Repository: github.com/HariomSThakur/Brandlens

The problem

Marketing teams reuse product facts, research results and approved claims across many drafts. Reviewers often have to search source documents manually to check exact wording, numbers and dates. Unsupported or outdated claims can slip through, and decisions become difficult to trace.

The solution

BrandLens gives small marketing and communications teams one evidence-linked workflow. A creator writes or imports a draft; deterministic checks compare it with approved sources, claims and writing rules; finding cards show the relevant text and evidence; a person resolves the findings and records approval. The app never publishes content automatically.

CORE JOURNEY

Draft to accountable decision

1. Write or import a text/Markdown draft. 2. Run deterministic checks and inspect evidence-linked findings. 3. Accept a verified edit or make a manual change. 4. Re-run review. 5. A Reviewer or Admin records a decision against the current draft and evidence versions.

The bundled Luma Botanics workspace is fictional. It demonstrates the numeric-conflict example: a claim that a result happened in 3 days is checked against a 28-day consumer test that measured hydration, not pigmentation.

Round 3 improvements delivered

The build extends the Round 2 workflow with evidence lifecycle management, policy feedback and safer revision suggestions.

IMPROVEMENT 1

Evidence-backed claim ledger and conflict detection

Each claim stores an owner, exact supporting excerpt, source ID and version, extracted key values with units, approved alternative wording, and a review date. Draft review flags numeric conflicts. Source changes, claim edits and expiry can queue dependent claims and previous approvals for a fresh human review. Approvals retain the claim and source versions they used; earlier records are preserved.

IMPROVEMENT 2

Guideline-health insights that close the loop

BrandLens groups repeated dismissals, accepted edits and escalations, and surfaces incompatible threshold guidance or a restricted term used by an approved claim. The page shows examples behind each signal. An Admin can record a decision to clarify a rule, add an exception or retire it, with a rationale. Follow-up dismissal rates appear when later outcomes exist. Policy is never edited automatically.

IMPROVEMENT 3

Verified minimal-diff rewrites

Suggestions use explicit terminology replacements from approved guidance or alternative wording already stored in the claim ledger. Optional AI tone suggestions are advisory. BrandLens protects links, names and numbers, checks claim values against the exact evidence excerpt, then re-runs deterministic checks. Each edit can be accepted or rejected separately; word-level differences and revision history are retained.

Completed product features

- Draft editor and plain text / Markdown import, with a 30,000-character limit.
- Findings for unsupported or expired claims, numeric conflicts, restricted terminology, all-caps text, repeated exclamation marks and long sentences.
- Source version history, evidence-backed claim ledger, re-review queue, guideline-health page and owner-decision tracking.
- Human approval records, local JSON backup, Light / Black / Dark blue themes, and Supabase Admin / Reviewer / Creator access code.

Technical implementation

The browser keeps the dependable review path local and deterministic. Optional infrastructure adds shared accounts and advisory model review without making either a prerequisite for the core workflow.

USER	APP	OPTIONAL SERVICES
Draft editor Evidence library Human decisions	HTML / CSS / vanilla JS Shared rule modules Finding and revision UI	Node.js 20 HTTP server Supabase Auth + Postgres Ollama / Qwen advisory path

Stack and data flow

- HTML, CSS and browser JavaScript ES modules; a small Node.js 20 server uses built-in modules and has no runtime package dependencies.
- Deterministic rules run in the browser against the draft, approved claims, source excerpts and dates. The review continues if a model or network connection fails.
- Optional Ollama/Qwen output is advisory. Exact draft quotes are checked, claim replacements cannot be generated by the model, and no AI output approves or publishes.
- Supabase Auth and Postgres store a shared JSONB workspace. Row-level security and role-checked RPCs enforce Admin, Reviewer and Creator access. An expected revision rejects stale saves with a reload message.
- Shared audit events use a separate append-only table. The database assigns event actor and timestamp; snapshot saves preserve the database audit history instead of accepting client edits to it.
- Render serves the public Node.js app. Local mode uses browser storage; hosted Qwen is not configured in this build, so the hosted review mode is deterministic unless an Ollama endpoint and model are configured.

RELIABILITY BOUNDARIES

A human remains responsible

Deterministic checks identify known patterns; they do not prove truth or guarantee that every claim is found. The database protects shared role boundaries and prevents stale snapshot overwrites. The append-only table prevents later update/delete of stored events, but it is not a cryptographic signature system. Apply the latest supabase/schema.sql before relying on the new shared audit stream.

Plan changes and verification

The Round 1 proposal was narrowed during implementation to prioritize the highest-risk, end-to-end workflow: evidence-linked claims, human review and traceable decisions.

Changes from the original plan

- The project uses a small Node.js server and vanilla browser modules rather than a heavier frontend framework, keeping the core workflow deployable with few dependencies.
- The Round 3 build adds the promised claim lifecycle, guideline-health insights and verified minimal rewrites.
- Plain text and Markdown import are implemented. DOCX parsing, image and alt-text checks, automated invitation email, workspace switching and multilingual review are deferred.
- The shared workspace remains a JSONB snapshot for this MVP; append-only audit events are separated into their own table, and a revision check blocks stale snapshot saves.

Testing notes

- npm test: 10 focused tests pass for quantity extraction, source and claim version invalidation, expiry queues, guideline conflicts, follow-up rates, safe rewrites, protected values and word diffs.
- npm run eval: 40 frozen synthetic drafts, including 15 clean drafts, score 100.0% micro precision and recall with 0 false alarms on the clean set. The project author wrote the rules, fixtures and labels; this is a self-check, not independent validation.
- Syntax checks pass for the browser modules and Node server. This environment denied loopback socket access, so automated local HTTP/browser smoke testing could not be completed.
- Live shared-account and audit-RPC tests require the owner to apply the latest Supabase schema and verify separate Reviewer and Creator accounts. User interviews completed: n=0.

KNOWN SCOPE LIMITS

What is not included

The demo uses fictional brand data. The hosted app has no Qwen checkpoint configured unless the owner adds an Ollama model endpoint. DOCX / image checking and automated invite emails are not implemented. These limits are disclosed so judges can assess the working scope accurately.

Team member contribution and AI disclosure

Hariom Thakur is the only team member. He defined the product direction and workflows, chose the core scope and role model, set up the Supabase / Render services, ran the project and validation scenarios, and prepared the submission. OpenAI Codex was used as a significant AI coding assistant for implementation, debugging and documentation. The participant should edit this contribution statement to match his own work and wording before filing the individual event form.

Official event page: Codex 48 - We Code Coders